

2026年春

程序设计实习：C++程序设计

第二讲 类和对象 (2)

贾川民
北京大学



□ 面向对象的基本概念

- 例子 — 矩形类
- 三种方式使用
- 引用 & 常引用
- 类成员的访问权限



主要内容

□ 面向对象的基本概念

- 例子—矩形类
- 对象的内存分配 与 运算符
- 三种方式使用
- 引用 & 常引用

□ 构造函数

□ 复制构造函数

□ 析构函数

构造函数 (Constructor)

- 在程序没有明确进行初始化的情况下，
 - 全局基本类型变量 — 被自动初始化成全0
 - 局部基本类型变量 — 初始值随机
- 构造函数是**成员函数**的一种，用来**初始化对象**
 - 名字与类名相同，可以有参数
 - 不能有返回值 (**注意: void也不行**)
 - 作用：为对象进行初始化，如给成员变量赋初值
 - 不必专门再写初始化函数，也不必担心忘记调用初始化函数
 - 避免对象未被初始化就使用而导致程序出错

构造函数 (Constructor)

- 如果定义类时没写构造函数
→ 编译器生成一个默认无参数的构造函数
 - 默认构造函数无参数, 不做任何操作
- 如果定义了构造函数, 则编译器不生成默认无参数的构造函数

对象生成时构造函数自动被调用, 对象一旦生成, 就再也不能在其上执行构造函数

- 为类编写构造函数是好的习惯, 能够保证对象生成的时候总是有合理的值
- 一个类可以有多个构造函数

构造函数 (Constructor)

```
class Complex {  
    private :  
        double real, imag;  
    public:  
        void Set( double r, double i );  
}; //编译器自动生成默认构造函数
```

`Complex c1;` //默认构造函数被调用

`Complex * pc = new Complex;` //默认构造函数被调用

构造函数 (Constructor)

```
class Complex {  
    private:  
        double real, imag;  
    public:  
        Complex(double r, double i = 0); //构造函数  
};  
Complex::Complex(double r, double i){  
    real = r;    imag = i;  
}  
Complex c1; // error, 没有参数  
Complex * pc = new Complex; // error, 没有参数  
Complex c2(2); // OK, Complex c2(2, 0)  
Complex c3(2, 4), c4(3, 5); //OK  
Complex * pc = new Complex(3, 4); //OK
```

构造函数 (Constructor)

- 可以有多个构造函数, 参数个数或类型不同

```
class Complex {  
    private :  
        double real, imag;  
    public:  
        void Set(double r, double i);  
        Complex(double r, double i);  
        Complex(double r);  
        Complex(Complex c1, Complex c2);  
};  
Complex::Complex(double r, double i){  
    real = r;    imag = i;  
}
```


构造函数 (Constructor)

```
Complex::Complex(double r)
```

```
{
```

```
    real = r; imag = 0;
```

```
}
```

```
Complex::Complex (Complex c1, Complex c2)
```

```
{
```

```
    real = c1.real+c2.real;
```

```
    imag = c1.imag+c2.imag;
```

```
}
```

```
Complex c1(3) , c2 (1, 0), c3(c1, c2);
```

```
// c1 = {3, 0}, c2 = {1, 0}, c3 = {4, 0};
```

构造函数 (Constructor)

- 构造函数最好是public的
- private构造函数不能直接用来初始化对象

```
class CSample{  
    private:  
        CSample() { }  
};
```

```
int main(){  
    CSample Obj;    //err. 唯一构造函数是private
```

构造函数在数组中的使用

```
class CSample {  
    int x;  
public:  
    CSample() {  
        cout << "Constructor 1 Called" << endl;  
    }  
    CSample(int n) {  
        x = n;  
        cout << "Constructor 2 Called" << endl;  
    }  
};
```

练习：构造函数在数组中的使用

```
int main(){
    CSample array1[2];
    cout << "step1"<<endl;
    CSample array2[2] = {4, 5};
    cout << "step2"<<endl;
    CSample array3[2] = {3};
    cout << "step3"<<endl;
    CSample * array4 = new CSample[2];
    delete []array4;
}
```

构造函数在数组中的使用

输出：

```
Constructor 1 Called  
Constructor 1 Called  
step1  
Constructor 2 Called  
Constructor 2 Called  
step2  
Constructor 2 Called  
Constructor 1 Called  
step3  
Constructor 1 Called  
Constructor 1 Called
```

构造函数在数组中的使用

```
class Test {  
    public:  
        Test(int n) { }           //(1)  
        Test(int n, int m) { }    //(2)  
        Test() { }                //(3)  
};  
Test array1[3] = {1, Test(1,2)};  
// 三个元素分别用(1), (2), (3)初始化  
Test array2[3] = { Test(2,3), Test(1,2) , 1};  
// 三个元素分别用(2), (2), (1)初始化  
Test * pArray[3] = {new Test(4), new Test(1,2)};  
//两个元素分别用(1), (2)初始化
```

例题改一改

```
class Test {  
    public:  
        Test (int n) { cout << n << "(1)" << endl; }           //(1)  
        Test (int n, int m) { cout << n << m << "(2)" << endl; } //(2)  
        Test ( ) { cout << "(3)" << endl; }                     //(3)  
};  
int main() {  
    Test array1[3] = { 1, Test(1, 2) };  
  
    Test array2[3] = { 1, (1, 2) };  
  
    return 0;  
}
```

例题改一改

```
class Test {  
    public:  
        Test (int n) { cout << n << "(1)" << endl; }           //(1)  
        Test (int n, int m) { cout << n << m << "(2)" << endl; } //(2)  
        Test ( ) { cout << "(3)" << endl; }                     //(3)  
};  
int main() {  
    Test array1[3] = { 1, Test(1, 2) };  
    // 三个元素分别用(1),(2),(3)初始化  
    Test array2[3] = { 1, (1, 2) };  
    // 三个元素分别用(1),(1),(3)初始化  
    return 0;  
}
```


例题改一改

```
class Test {  
    public:  
        Test (int n) { cout << n << "(1)" << endl; }           //(1)  
        Test (int n, int m) { cout << n << m << "(2)" << endl; } //(2)  
        Test ( ) { cout << "(3)" << endl; }                     //(3)  
};  
  
int main() {  
    Test array1[3] = { 1, Test(1, 2) };  
    // 三个元素分别用(1),(2),(3)初始化  
    Test array2[3] = { 1, (1, 2) };  
    // 三个元素分别用(1),(1),(3)初始化  
    return 0;  
}
```

主要内容

□ 面向对象的基本概念

- 例子——矩形类
- 对象的内存分配 与 运算符
- 三种方式使用
- 引用 & 常引用

□ 构造函数

□ 复制构造函数

□ 析构函数



复制构造函数 (拷贝构造函数)

- 只有一个参数, 即对同类对象的引用
 - 形如 `X::X( X& )` 或 `X::X( const X& )` 二者选一, 后者能以常量对象作为参数

复制构造函数 (拷贝构造函数)

- 只有一个参数, 即对同类对象的引用
 - 形如 **`X::X( X& )`** 或 **`X::X( const X& )`** 二者选一, 后者能以常量对象作为参数
- 如果没有定义复制构造函数, 那么编译器生成默认复制构造函数
 - 默认的复制构造函数完成复制功能
- 如果定义的自己的复制构造函数, 则默认的复制构造函数不存在。
- 不允许有形如 `X::X( X )` 的构造函数

复制构造函数 (拷贝构造函数)

- 只有一个参数, 即对同类对象的引用

```
class Complex {  
    private :  
        double real, imag;  
};
```

```
Complex c1;           //调用缺省无参构造函数  
Complex c2(c1);       //调用缺省的复制构造函数  
                      //将 c2 初始化成和c1一样
```

□ 如果定义的自己的复制构造函数, 则缺省的复制构造函数不存在

```
class Complex {  
    public :  
        double real, imag;  
        Complex(){}  
        Complex(Complex & c) {  
            real = c.real;  
            imag = c.imag;  
            cout << "Copy Constructor called";  
        }  
};  
  
Complex c1;  
Complex c2(c1); //调用自己定义的复制构造函数,
```



复制构造函数 (拷贝构造函数)

- 不允许有形如 $X::X(X)$ 的构造函数

```
class CSample {  
    CSample(CSample c) {  
    } //错, 不允许这样的构造函数  
};
```

复制构造函数 (拷贝构造函数)

复制构造函数在以下三种情况被调用:

a. 当用一个对象去初始化同类的另一个对象时

```
Complex c2(c1);
```

```
Complex c2 = c1; //初始化语句, 非赋值语句
```


b.如果某函数有一个参数是类 A 的对象,那么该函数被调用时,类A的复制构造函数将被调用

```
class A {  
    public:  
        A() { };  
        A(A & a) {  
            cout << "Copy constructor called" << endl;  
        }  
};  
void Func(A c){ }  
int main(){  
    A c1;  
    Func(c1);  
    return 0;  
}
```

程序输出结果为:
Copy constructor called

复制构造函数 (拷贝构造函数)

c. 如果函数的返回值是类A的对象时, 则函数返回时,
A的复制构造函数被调用:

```
class A {  
    public:  
        int v;  
        A(int n) { v = n; };  
        A( const A & a) {  
            v = a.v;  
            cout << "Copy constructor called" << endl;  
        }  
};  
  
A Func() { A b(4); return b; }  
int main(){ cout << Func().v << endl; return 0; }
```

输出结果:

Copy constructor called
4

经过优化的编译器可能导致结果不同

复制构造函数 (拷贝构造函数)

[注意]:对象间用等号赋值并不导致复制构造函数被调用

```
class CMyclass {  
    public:  
        int n;  
        CMyclass() { }; //构造函数  
        CMyclass(CMyclass & c) { n = 2 * c.n; } //复制构造函数  
};  
int main() {  
    CMyclass c1, c2;  
    c1.n = 5;    c2 = c1;    CMyclass c3(c1);  
    cout << "c2.n=" << c2.n << ", ";  
    cout << "c3.n=" << c3.n << endl;  
    return 0;  
} // 输出: c2.n=5, c3.n=10
```

常量引用参数的使用

```
void fun(CMyclass obj_ ) {  
    cout << "fun" << endl;  
}
```

- 调用时生成形参会引发复制构造函数调用, 开销比较大
- 所以可以考虑使用 **CMyclass & 引用类型** 作为参数
- 希望确保实参的值在函数中不应被改变
→ 可以加上 const 关键字:

```
void fun(const CMyclass & obj) {  
    //函数中任何试图改变 obj值的语句都将是变成非法  
}
```

类型转换构造函数

□ 类型转换构造函数

- **目的:** 实现类型的自动转换
- 只有一个参数
- 又不是复制构造函数的构造函数
 - 该构造函数起到类型自动转换的作用
- 需要时编译系统会自动调用转换构造函数
 - 建立一个无名的临时对象 (或临时变量)

类型转换构造函数实例

```
class Complex {  
    public:  
        double real, imag;  
        Complex(int i) { //类型转换构造函数  
            cout << "Int Constructor called" << endl;  
            real = i; imag = 0;  
        }  
        Complex(double r, double i) { real = r; imag = i; }  
};  
int main (){  
    Complex c1(7, 8);  
    Complex c2 = 12;  
    c1 = 9; // 9被自动转换成一个临时Complex对象  
    cout << c1.real << "," << c1.imag << endl;  
    return 0;  
}
```

输出:

```
IntConstructor called  
IntConstructor called  
9, 0
```

类型转换构造函数实例

```
class Complex {  
    public:  
        double real, imag;  
        explicit Complex(int i) { //显式类型转换构造函数  
            cout << "Int Constructor called" << endl;  
            real = i; imag = 0;  
        }  
        Complex(double r, double i) { real = r; imag = i; }  
};  
int main (){  
    Complex c1(7, 8);  
    Complex c2 = Complex(12);  
    c1 = 9; // error, 9不被自动转换成一个临时Complex对象  
    c1 = Complex(9); //ok  
    cout << c1.real << ", " << c1.imag << endl;  
    return 0;  
}
```

explicit 关键字的作用就是防止类构造函数的隐式自动转换

析构函数 (Destructors)

□ 成员函数的一种

- 名字与类名相同, 在前面加 ‘~’
 - 没有参数和返回值
 - 一个类最多只有一个析构函数
- 析构函数对象消亡时 → 自动被调用
 - 定义析构函数 → 对象消亡前做善后工作
e.g. 释放分配的空间
- 如果定义类时没写析构函数, 则编译器生成缺省析构函数
 - 缺省析构函数什么也不做
- 如果定义了析构函数, 则编译器不生成缺省析构函数

析构函数 (Destructors)

```
class CString{  
    private :  
        char * p;  
    public:  
        CString () {  
            p = new char[10];  
        }  
        ~ CString () ;  
};  
CString::~~ CString()  
{  
    delete [] p;  
}
```

析构函数和数组

- 对象数组生命期结束时，
对象数组的每个元素的析构函数都会被调用

```
class Ctest {  
    public:  
        ~Ctest() { cout<< "destructor called" << endl; }  
};  
  
int main () {  
    Ctest array[2];  
    cout << "End Main" << endl;  
    return 0;  
}
```

输出：
End Main
destructor called
destructor called

析构函数和数组

- delete 运算导致析构函数调用

```
Ctest * pTest;  
pTest = new Ctest;           //构造函数调用  
delete pTest;                //析构函数调用
```

```
pTest = new Ctest[3]; //构造函数调用3次  
delete [] pTest;      //析构函数调用3次
```

若new一个对象数组, 那么用delete释放时应该写 []
否则只delete一个对象(调用一次析构函数)

析构函数在对象作为函数返回值返回后被调用

```
class CMyclass {  
    public:  
        ~CMyclass() { cout << "destructor" << endl; }  
};  
CMyclass obj;  
CMyclass fun(CMyclass sobj ) { //参数对象消亡也会导致析  
                                //构函数被调用  
    return sobj;    //函数调用返回时生成临时对象返回  
}  
int main(){  
    obj = fun(obj); //函数调用的返回值 (临时对象) 被用过后,  
    return 0;      //该临时对象析构函数被调用  
}
```

```

class CMyclass {
    public:
        ~CMyclass() { cout << "destructor" << endl; }
};
CMyclass obj;
CMyclass fun(CMyclass sobj ) { //参数对象消亡也会导致析
                                //构造函数被调用
    return sobj;    //函数调用返回时生成临时对象返回
}
int main(){
    obj = fun(obj); //函数调用的返回值 (临时对象) 被用过后,
    return 0;       //该临时对象析构造函数被调用
}

```

输出:

destructor
destructor
destructor

在临时对象生成的时候会有构造函数被调用;
临时对象消亡导致析构造函数调用

构造函数和析构函数
什么时候被调用？

```
class Demo {  
    int id;  
public:  
    Demo(int i) {  
        id = i;  
        printf( "id=%d, Construct\n", id);  
    }  
    ~Demo() {  
        printf( "id=%d, Destruct\n", id);  
    }  
};
```

```
Demo d1(1);
void fun(){
    static Demo d2(2);
    Demo d3(3);
    printf( "fun \n");
}
int main (){
    Demo d4(4);
    printf( "main \n");
    { Demo d5(5); }
    fun();
    printf( "endmain \n");
    return 0;
}
```



```
Demo d1(1);  
void fun(){  
    static Demo d2(2);  
    Demo d3(3);  
    printf( "fun \n");  
}  
int main (){  
    Demo d4(4);  
    printf( "main \n");  
    { Demo d5(5); }  
    fun();  
    printf( "endmain \n");  
    return 0;
```

输出：

id=1, Construct
id=4, Construct
main
id=5, Construct
id=5, Destruct
id=2, Construct
id=3, Construct
fun
id=3, Destruct
endmain
id=4, Destruct
id=2, Destruct
id=1, Destruct

关于复制构造函数和析构函数的又一个例子

```
#include <iostream>
using namespace std;
class CMyclass {
public:
    CMyclass() {};
    CMyclass( CMyclass & c )
    {
        cout << "copy constructor" << endl;
    }
    ~CMyclass() { cout << "destructor" << endl; }
};
```



关于复制构造函数和析构函数的又一个例子

```
void fun(CMyclass obj_ ) {  
    cout << "fun" << endl;  
}  
CMyclass c;  
CMyclass Test( ) {  
    cout << "test" << endl;  
    return c;  
}  
int main(){  
    CMyclass c1;  
    fun(c1);  
    Test();  
    return 0;  
}
```

```
void fun(CMyclass obj_ ) {  
    cout << "fun" << endl;  
}  
CMyclass c;  
CMyclass Test( ) {  
    cout << "test" << endl;  
    return c;  
}  
int main(){  
    CMyclass c1;  
    fun(c1);  
    Test();  
    return 0;  
}
```

运行结果:

copy constructor

fun

destructor //参数消亡

test

copy constructor

destructor // 返回值临时对象消亡

destructor // 局部变量消亡

destructor // 全局变量消亡

复制构造函数在不同编译器中的表现

```
1  class A {  
2  public:  
3      int x;  
4      A(int x_):x(x_) {  
5          cout << x << " constructor called" << endl;  
6      }  
7      A(const A & a ) {  
8          x = 2 + a.x;  
9          cout << "copy called" << endl;  
10     }  
11     ~A() {  
12         cout << x << " destructor called" << endl;  
13     }  
14 };  
15  
16 A f() {  
17     A b(10);  
18     return b;  
19 }  
20 int main() {  
21     A a(1);  
22     a = f();  
23     return 0;  
24 }
```

某些版本 Visual
Studio 输出结果:
1 constructor called
10 constructor called
10 destructor called
copy called
12 destructor called
12 destructor called

复制构造函数在不同编译器中的表现

```
1  class A {  
2  public:  
3      int x;  
4      A(int x_):x(x_) {  
5          cout << x << " constructor called" << endl;  
6      }  
7      A(const A & a ) {  
8          x = 2 + a.x;  
9          cout << "copy called" << endl;  
10     }  
11     ~A() {  
12         cout << x << " destructor called" << endl;  
13     }  
14 };  
15  
16 A f() {  
17     A b(10);  
18     return b;  
19 }  
20 int main() {  
21     A a(1);  
22     a = f();  
23     return 0;  
24 }
```

DEVCCPP 输出结果:

1 constructor called
10 constructor called
10 destructor called
10 destructor called

复制构造函数在不同编译器中的表现

- 某些编译器出于优化目的并未生成返回值临时对象



复制构造函数在以下三种情况被调用：

- 1) 当用一个对象去初始化同类的另一个对象时
- 2) 如果某函数有一个参数是类A的对象，那么该函数被调用时，类A的复制构造函数将被调用
- 3) 如果函数的返回值是类A的对象时，则函数返回时，A的复制构造函数被调用

类和对象主要内容

- 内联函数/内联成员函数
- 成员函数重载与参数缺省
- this指针
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元

课程主要内容

- 内联函数/内联成员函数
- 成员函数重载与参数缺省
- this 指针
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元

- 函数调用本身是有时间开销的
- 如果函数本身只有几条语句, 执行非常快

Vs. 但是函数被反复执行很多次, 调用函数所产生的这个开销就会显得比较大



内联函数

- 定义函数时, 在返回值类型前面加 **inline** 关键字, 可以使得函数成为 内联函数
- 编译器处理内联函数的调用语句时 → 将整个函数的代码插入到调用语句处, 而不会生出调用函数的语句

```
inline int Max(int a, int b)
{
    if( a > b ) return a;
    return b;
}
```

内联成员函数

- 在成员函数前面加上 **inline** 关键字后, 成员函数就成为 内联成员函数
- 将整个函数体写在类定义内部, 函数也会成为 内联成员函数

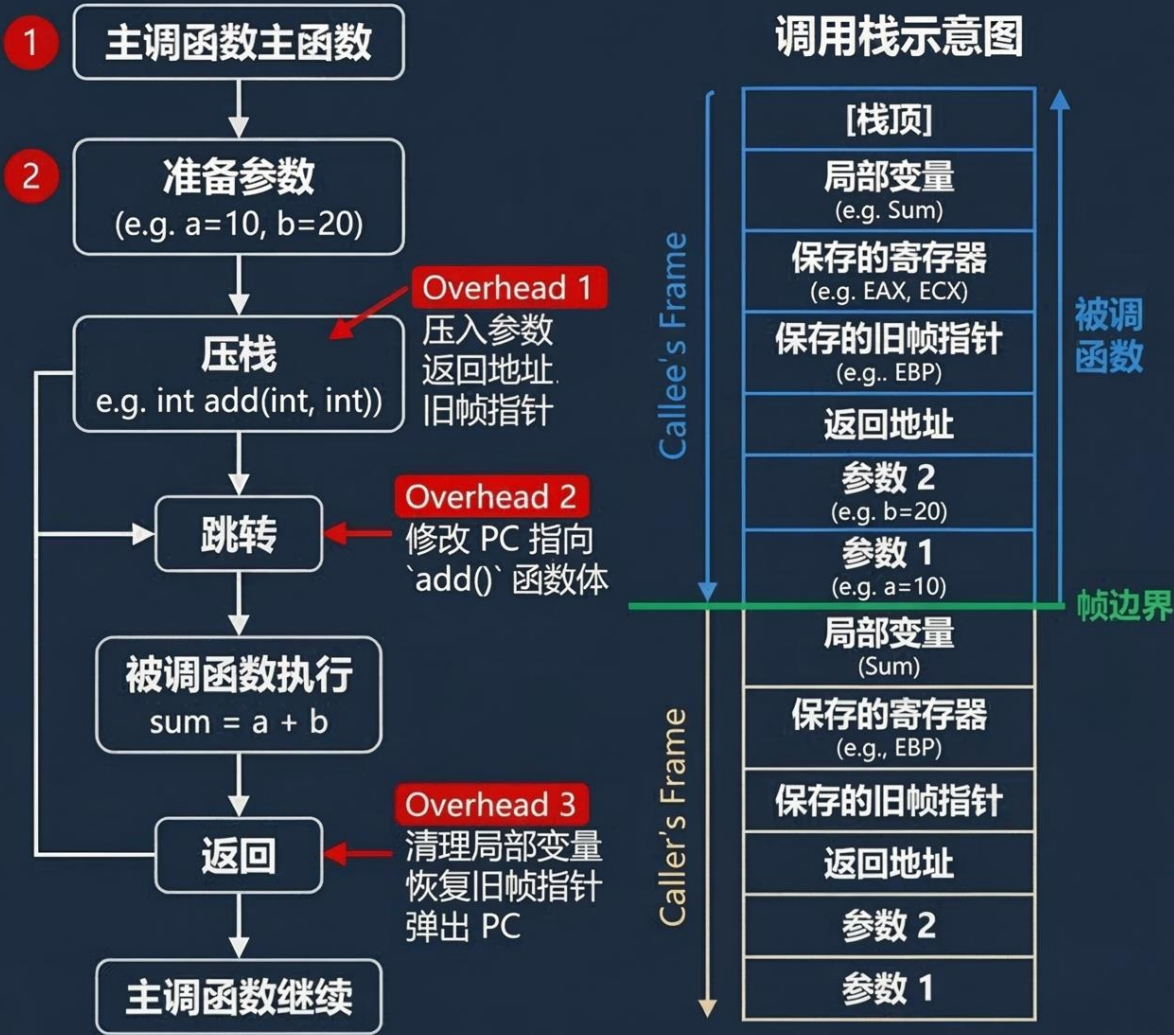
```
class B{  
    inline void func1();  
    void func2() { }  
};
```

```
void B::func1() { }
```

func1和func2都是内联成员函数

C++ 内联函数调用开销与inline总结图

C++ 函数调用开销 (Function Call Overhead)



'inline' 的优点与缺点总结

优点	缺点
1. 消除函数调用开销: 避免压栈、跳转、返回, 适合小且频繁调用的函数。	1. 代码膨胀 (Code Bloat): 在每个调用点复制函数, 增加二进制文件大小, 加重指令缓存 (I-cache) 负担。
2. 提高执行速度: 代码直接插入, 减少指令数量, 提高 ILP (Instruction-Level Parallelism)。	2. 增加编译时间: 编译器需要处理更多、更复杂的代码扩展。
3. 增强优化机会: 允许编译器跨函数边界进行优化, 如常量折叠、死代码消除。	3. 调试困难: 在调试时, 内联函数没有独立的栈帧, 单步跟踪更复杂。
4. 类型安全与作用域: 相比 C 宏, 'inline' 是真正的函数, 拥有完整类型检查。 <pre>void main() { int res = add(1, 2); }</pre>	4. 非强制性 (Not Guaranteed): 'inline' 只是建议, 复杂函数 (如含循环、递归、过大) 常被编译器忽略。 <pre>void main() { int res = (1 + 2); }</pre>

主要内容

- 内联函数/内联成员函数
- 成员函数重载与参数缺省
- this 指针
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元

函数参数的默认值

□ 任何有定义的表达式都可以成为函数参数的默认值:

```
int Max(int m, int n);
```

```
int a, b;
```

```
void Function2 ( int x, int y = Max(a, b), int z = a*b) {
```

```
...
```

```
}
```

```
Function2(4);
```

```
// 正确, 等效于 Function2(4, Max (a, b), a*b);
```

```
Function2(4, 9); // 正确, 等效于 Function2(4, 9, a*b);
```

```
Function2(4, 2, 3); // 当然正确
```

```
Function2(4, , 3); // 错误! 这样的写法不允许,
```

```
// 省略的参数一定是最右边连续的几个
```


成员函数的重载及参数默认

- 成员函数也可以重载 (普通函数也可以)
- 成员函数/构造函数可以带默认值参数 (普通函数也可以)

```
#include <iostream>
using namespace std;
class Location {
    private :
        int x, y;
    public:
        void init( int x=0 , int y = 0 );
        void valueX( int val ) { x = val; }
        int valueX() { return x; }
};
void Location::init( int X, int Y) {
    x = X; y = Y;
}
```

```
int main() {  
    Location A, B;  
    A.init(5);  
    A.valueX(5);  
    cout << A.valueX();  
    return 0;  
}
```

□ 使用缺省参数要注意避免有函数重载时的二义性

```
class Location {  
    private:  
        int x, y;  
    public:  
        void init( int x =0, int y = 0 );  
        void valueX( int val = 0 ) { x = val; }  
        int valueX() { return x; }  
};
```

Location A;

A.valueX(); //错误, 编译器无法判断调用哪个valueX

主要内容

- 内联函数/内联成员函数
- 成员函数重载与参数缺省
- **this** 指针
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元



C++ 程序到C程序的翻译

```
class CCar {  
    public:  
        int price;  
        void SetPrice(int p);  
};  
  
void CCar::SetPrice(int p)  
{    price = p; }  
  
int main(){  
    CCar car;  
    car.SetPrice(20000);  
    return 0;  
}
```

C++ 程序到C程序的翻译

```
class CCar {  
    public:  
        int price;  
        void SetPrice(int p);  
};  
void CCar::SetPrice(int p)  
{    price = p; }  
int main(){  
    CCar car;  
    car.SetPrice(20000);  
    return 0;  
}
```

C++ 程序到C程序的翻译

```
class CCar {  
    public:  
        int price;  
        void SetPrice(int p);  
};  
void CCar::SetPrice(int p)  
{    price = p; }  
int main(){  
    CCar car;  
    car.SetPrice(20000);  
    return 0;  
}
```

```
struct CCar {  
    int price;  
};
```



C++ 程序到C程序的翻译

```
class CCar {  
    public:  
        int price;  
        void SetPrice(int p);  
};  
void CCar::SetPrice(int p)  
{    price = p; }  
int main(){  
    CCar car;  
    car.SetPrice(20000);  
    return 0;  
}
```

```
struct CCar {  
    int price;  
};  
void SetPrice(struct CCar * this, int p)  
{    this->price = p; }
```


C++ 程序到C程序的翻译

```
class CCar {  
    public:  
        int price;  
        void SetPrice(int p);  
};  
void CCar::SetPrice(int p)  
{    price = p; }  
int main(){  
    CCar car;  
    car.SetPrice(20000);  
    return 0;  
}
```

```
struct CCar {  
    int price;  
};  
void SetPrice(struct CCar * this, int p)  
{    this->price = p; }  
int main() {  
    struct CCar car;  
    SetPrice( & car, 20000 );  
    return 0;  
}
```

this 指针作用

- 其作用就是指向成员函数所作用的**对象**

this指针的作用

```
class A {  
    int i;  
    public:  
        void Hello() { cout << "hello" << endl; }  
};  
  
int main() {  
    A * p = NULL;  
    p->Hello(); //结果会怎样?  
}
```

this指针的作用

```
class A {  
    int i;  
    public:  
        void Hello() { cout << "hello" << endl; }  
};  
  
int main() {  
    A * p = NULL;  
    p->Hello();  
} // 输出: hello
```

this指针的作用

```
class A {  
    int i;  
    public:  
        void Hello() { cout << "hello" << endl; }  
}; // void Hello( A * this ) { cout << "hello" << endl; }
```

```
int main() {  
    A * p = NULL;  
    p->Hello();  
} // 输出: hello
```

this指针的作用

```
class A {  
    int i;  
    public:  
        void Hello() { cout << "hello" << endl; }  
}; // void Hello( A * this ) { cout << "hello" << endl; }
```

```
int main() {  
    A * p = NULL;  
    p->Hello(); // Hello(p);  
} // 输出: hello
```

this指针的作用

```
class A {  
    int i;  
    public:  
        void Hello() { cout << i << "hello" << endl; }  
};
```

```
int main() {  
    A * p = NULL;  
    p->Hello(); // Hello(p);  
}
```

this指针的作用

```
class A {  
    int i;  
    public:  
        void Hello() { cout << i << "hello" << endl; }  
}; // void Hello(A * this ) { cout << this->i << "hello" << endl; }  
  
int main() {  
    A * p = NULL;  
    p->Hello(); // Hello(p);  
}
```


this指针的作用

```
class A {  
    int i;  
    public:  
        void Hello() { cout << i << "hello" << endl; }  
}; // void Hello(A * this ) { cout << this->i << "hello" << endl; }  
    // this若为NULL, 则出错  
int main() {  
    A * p = NULL;  
    p->Hello(); // Hello(p);  
}
```

this指针的作用

非静态成员函数中可以直接使用**this**来代表指向该函数作用的对象的指针

```
class Complex {  
public:  
    double real, imag;  
    void Print() {    cout << real << "," << imag;    }  
    Complex(double r, double i):real(r), imag(i) {    }  
    Complex AddOne() {  
        this->real ++; //等价于 real ++;  
        this->Print(); //等价于 Print()  
        return * this;  
    }  
};
```

```
int main() {  
    Complex c1(1,1), c2(0,0);  
    c2 = c1.AddOne();  
    return 0;  
}
```

this指针的作用

非静态成员函数中可以直接使用**this**来代表指向该函数作用的对象的指针

```
class Complex {  
public:  
    double real, imag;  
    void Print() {    cout << real << "," << imag;    }  
    Complex(double r, double i):real(r), imag(i) {    }  
    Complex AddOne() {  
        this->real ++; //等价于 real ++;  
        this->Print(); //等价于 Print()  
        return * this;  
    }  
};
```

```
int main() {  
    Complex c1(1,1), c2(0,0);  
    c2 = c1.AddOne();  
    return 0;  
} //输出 2,1
```

主要内容

- 内联函数/内联成员函数
- 成员函数重载与参数缺省
- this 指针
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元



静态成员变量和静态成员函数

- 静态成员：在说明前面加了 **static** 关键字的成员

```
class CRectangle {  
private:  
    int w, h;  
    static int nTotalArea; //静态成员变量  
    static int nTotalNumber;  
public:  
    CRectangle(int w_, int h_);  
    ~CRectangle();  
    static void PrintTotal(); //静态成员函数  
};  
CRectangle r;
```

静态成员变量和静态成员函数

□ 静态成员

- 普通成员变量每个对象有各自的一份

Vs. 静态成员变量一共就一份, 为所有对象共享

- sizeof 运算符不会计算静态成员变量



静态成员变量和静态成员函数

sizeof 运算符不会计算静态成员变量

```
class CMyclass {  
    int n;  
    static int s;  
};
```

则 `sizeof(CMyclass)` 等于 4

静态成员变量和静态成员函数

□ 静态成员

- 普通成员变量每个对象有各自的一份

Vs. 静态成员变量一共就一份, 为所有对象共享

- sizeof 运算符不会计算静态成员变量

- 普通成员函数必须具体作用于某个对象

Vs. 静态成员函数并不具体作用于某个对象

- 静态成员 **不需要通过对象就能访问**



静态成员变

访问静态成员方式:

(1) 通过:

类名::成员名 的方式: **CRectangle::PrintTotal();**

(2) 也可以和普通成员一样采取

- 对象名.成员名 **r.PrintTotal();**

- 指针->成员名 **CRectangle * p = &r;**
p->PrintTotal();

- 引用.成员名 **CRectangle & ref = r;**
int n = ref.nTotalNumber;

上面这三种方式, 效果和**类名::成员名**没区别

静态成员变量不会属于某个特定对象

静态成员函数不会作用于某个特定对象

静态成员变量和静态成员函数

- 静态成员变量本质上是全局变量
 - 哪怕一个对象都不存在，类的静态成员变量也存在
- 静态成员函数本质上是全局函数
- 设置静态成员这种机制的 目的
 - 将与某些类紧密相关的全局变量和函数写到类里面
 - 看上去像一个整体
 - 易于维护和理解

静态成员变量和静态成员函数

- 设计一个需随时知道矩形总数和总面积的图形处理程序
 - 可以用全局变量来记录这两个值

Vs. 用静态成员封装进类中, 就更容易理解和维护

```
class CRectangle{  
    private:  
        int w, h;  
        static int nTotalArea;  
        static int nTotalNumber;  
    public:  
        CRectangle(int w_, int h_);  
        ~CRectangle();  
        static void PrintTotal();  
};
```

```
CRectangle::CRectangle(int w_, int h_){  
    w = w_;  
    h = h_;  
  
    nTotalNumber ++;  
    nTotalArea += w * h;  
}  
  
CRectangle::~~CRectangle(){  
    nTotalNumber --;  
    nTotalArea -= w * h;  
}  
  
void CRectangle::PrintTotal(){  
    cout << nTotalNumber << ", " << nTotalArea << endl;  
}
```



静态成员变量和静态成员函数

```
int CRectangle::nTotalNumber = 0;
```

```
int CRectangle::nTotalArea = 0;
```

```
// 必须在定义类定义的外面专门对静态成员变量进行声明
```

```
// 同行可以初始化; 否则编译能通过, 链接不能通过
```

```
int main()
```

```
{
```

```
    CRectangle r1(3, 3), r2(2, 2);
```

```
    //cout << CRectangle::nTotalNumber; // Wrong, 私有
```

```
    CRectangle::PrintTotal();
```

```
    r1.PrintTotal();
```

```
    return 0;
```

```
}
```

输出结果:

2, 13

2, 13

静态成员变量和静态成员函数

- 在静态成员函数中，
 - 不能访问**非静态**成员变量
 - 不能调用**非静态**成员函数

```
void CRectangle::PrintTotal(){  
    cout << w << ", " << nTotalNumber << ", " << nTotalArea << endl;  
    //wrong  
}
```

因为：

```
CRectangle r;
```

```
r.PrintTotal(); // 解释得通
```

```
CRectangle::PrintTotal(); //解释不通, w 到底是属于那个对象的?
```

静态成员函数中不能使用**this**指针
因为静态成员函数并不具体作用于某个对象
→ 静态成员函数的真实参数个数, 就是程序中写出的参数个数

复制构造函数和静态变量

```
class CRectangle
{
    private:
        int w, h;
        static int nTotalArea;
        static int nTotalNumber;
    public:
        CRectangle(int w_, int h_);
        ~CRectangle();
        static void PrintTotal();
};
```

复制构造函数和静态变量

```
CRectangle::CRectangle(intw_, int h_){  
    w = w_;  
    h = h_;  
    nTotalNumber ++;  
    nTotalArea += w * h;  
}  
CRectangle::~~CRectangle(){  
    nTotalNumber --;  
    nTotalArea -= w * h;  
}  
void CRectangle::PrintTotal(){  
    cout << nTotalNumber << ", " << nTotalArea << endl;  
}
```


复制构造函数和静态变量

□ 现有设计的CRectangle类的不足之处:

■ 在使用中, 如果调用复制构造函数

→ 临时隐藏的CRectangle对象

□ 调用一个以CRectangle类对象作为参数的函数时

□ 调用一个以CRectangle类对象作为返回值的函数

■ 则临时对象在消亡时 → 调用析构函数

□ 减少nTotalNumber 和 nTotalArea的值

□ 但临时对象生成时, 却没有增加 nTotalNumber 和 nTotalArea的值

复制构造函数和静态变量

- 要为CRectangle类写一个复制构造函数

```
CRectangle :: CRectangle(CRectangle & r ){
```

```
    w = r.w;  h = r.h;
```

```
    nTotalNumber ++;
```

```
    nTotalArea += w * h;
```

```
}
```

主要内容

- 内联函数/内联成员函数
- 成员函数重载与参数缺省
- this 指针
- 静态成员
- 常量成员函数**
- 成员对象和封闭类
- 友元



复习: const 的用法

- 对指针定义**const**, 则不可通过该指针修改其指向地方的内容

```
int n, m;
```

```
const int * p = & n;
```

```
* p = 5; //编译出错
```

```
n = 4; //ok
```

```
p = &m; //ok
```

注意: 不能把常量指针赋值给非常量指针, 反过来可以

```
const int * p1; int * p2;
```

```
p1 = p2; //ok
```

```
p2 = p1; //error
```

```
p2 = (int * ) p1; //ok
```

复习: const 的用法

- 不希望在函数内部不小心写了改变参数指针所指地方的内容的语句 → 使用常数指针参数

```
void MyPrintf ( const char * p ){  
    strcpy( p, "this"); //编译出错  
    cout << p;         //ok  
}
```

- 用在引用上

```
int n;  
const int & r = n;  
r = 5;    //error  
n = 4;    //ok
```

常量对象和常量方法

□ 如果不希望某个对象的值被改变

→ 则定义该对象的时候可以在前面加 **const**关键字

```
class Sample {  
    private:  
        int value;  
    public:  
        void GetValue() {    }  
};  
  
const Sample Obj; // 常量对象  
Obj.GetValue();   // 错误
```

常量对象只能使用:

构造函数, 析构函数 和 有const说明的函数 (常量方法)

- 在类的成员函数说明后面加**const**关键字, 则该成员函数成为**常量成员函数**
- 常量成员函数内部不能改**非静态属性的值**, 也不能调用同类的非常量成员函数 (**静态成员函数除外**)

```
class Sample {  
    public: int value;  
        void GetValue( ) const;  
        void func( ) { }  
};  
void Sample::GetValue( ) const {  
    value = 0; // wrong  
    func();    //wrong  
}  
int main(){  
    const Sample o;  
    o.GetValue( ); //常量对象上可以执行常量成员函数  
    return 0;  
} // Visual Studio 2010 中没有问题,  
    // 在Dev C++中, 要为Sample类编写无参构造函数才可以
```

□ 在定义常量成员函数和声明常量成员函数时都应该使用const关键字

```
class Sample {  
    private :  
        int value;  
    public:  
        void GetValue() const;  
};
```

```
void Sample::GetValue() const {  
    cout << value;  
}
```

//此处不使用const会
//导致编译出错



Const的用法

- 如果觉得传递一个对象效率太低 (导致复制构造函数调用, 还费时间) 又不想传递指针, 又要确保实际参数的值不能在函数中被修改, 那么可以使用:

const T & 类型的参数

```
void PrintfObj( const Sample & o ){  
    o.func();           //error  
    o.GetValue();       //ok  
    Sample & r = (Sample &) o; //必须强制类型转换  
    r.func();           //ok  
}
```

常量成员函数的重载

□ 两个函数, 名字和参数表都一样

但是一个是const, 一个不是, 算重载



常量对象和常量方法

```
class CTest {  
    private :  
        int n;  
    public:  
        CTest() { n = 1 ; }  
        int GetValue() const { return n ; }  
        int GetValue() { return 2 * n ; }  
};  
int main() {  
    const CTest objTest1;  
    CTest objTest2;  
    cout << objTest1.GetValue() << ", " <<  
    objTest2.GetValue();  
    return 0;  
}
```

输出结果: 1, 2

mutable成员变量

- 可以在const成员函数中修改的成员变量

```
class CTest
{
    public:
        bool GetData() const
        {
            m_n1++;
            return m_b2;
        }
    private:
        mutable int m_n1; //使用mutable关键字
        bool m_b2;
};
```

主要内容

- 内联函数/内联成员函数
- 成员函数重载与参数缺省
- this 指针
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元



成员对象和封闭类

- **成员对象**: 一个类的成员变量是另一个类的对象
- 有**成员对象**的类叫封闭类 (enclosing)

```
class CTyre {           //轮胎类
    int radius;         //半径
    int width;          //宽度
public:
    CTyre(int r, int w):radius(r), width(w) { }
};
```

```
class CEngine {         //引擎类
};
```

成员对象和封闭类

```
class CCar { //汽车类
    int price; //价格
    CTyre tyre;
    CEngine engine;
public:
    CCar( int p, int tr, int tw );
};
CCar::CCar( int p, int tr, int tw ): price(p), tyre(tr, tw)
{
};
int main()
{
    CCar car(20000, 17, 225);
    return 0;
}
```

成员对象和封闭类

- 如果 CCar类不定义构造函数，则：

```
CCar car;    // compile error
```

因为编译器不明白 **car.tyre**该如何初始化；

car.engine 的初始化没问题，用默认构造函数即可

- 任何生成封闭类对象的语句，都要让编译器明白

→ 对象中的成员对象，是如何初始化的

- 具体的做法就是

通过**封闭类的构造函数的初始化列表**

封闭类构造函数的初始化列表

- 定义封闭类的构造函数时, 添加**初始化列表**:

```
类名::构造函数(参数表):成员变量1(参数表), 成员变量2(参数表), ...  
{  
    ...  
}
```

- 成员对象初始化列表中的参数

- 任意复杂的表达式
- 函数 / 变量 / 表达式中的函数, 变量有定义

封闭类构造函数的初始化列表

- 封闭类对象生成时
 - 先执行所有**成员对象**的构造函数
 - 然后才执行**封闭类**的构造函数
- 对象成员的构造函数调用次序和对象成员在**类中的说明次序**一致
- 与它们在**成员初始化列表**中出现的次序无关
- 封闭类的对象消亡时
 - 先执行**封闭类**的析构函数
 - 然后再执行**成员对象**的析构函数
- 次序和构造函数的调用次序相反

封闭类例子程序

```
class CTyre{           //轮胎类
    int radius;        //半径
    int width;         //宽度
public:
    CTyre( int r, int w ): radius(r), width(w) { cout << "CTyre
contructor" << endl; }
    ~CTyre() { cout << "CTyre destructor" << endl; }
};
class CEngine {        //引擎类
    int engNum;
public:
    CEngine( int n ) { engNum = n; cout << "CEngine
contructor" << endl; }
    ~CEngine() { cout << "CEngine destructor" << endl; }
};
```

封闭类例子程序

```
class CCar { //汽车类
    int price; //价格
    CEngine engine;
    CTyre tyre;

public:
    CCar( int p, int tr, int tw, int eNum );
    ~CCar() { cout << "CCar destructor" << endl; }
};

CCar::CCar( int p, int tr, int tw, int eNum ): price(p), tyre(tr,
tw), engine(eNum)
{
    cout << "CCar constructor" << endl;
};
```

封闭类例子程序

```
int main(){  
    car(20000, 17, 225, 100);  
    return 0;  
}
```

程序的输出结果是:

CEngine contructor

CTyre contructor

CCar constructor

CCar destructor

CTyre destructor

CEngine destructor

封闭类的复制构造函数

- 封闭类的对象, 如果是用默认复制构造函数初始化
→ 它里面包含的成员对象, 也会用复制构造函数初始化

```
class A{  
    public:  
        A() { cout << "default" << endl; }  
        A(A & a) { cout << "copy" << endl; }  
};  
class B { A a; };  
int main(){  
    B b1, b2(b1);  
    return 0;  
}
```

输出结果:
default
copy

- 说明**b2.a**是用类A的复制构造函数初始化
- 而调用复制构造函数时的**实参**就是b1.a

其他特殊成员: const成员和引用成员

- 初始化const成员和引用成员时, 必须在成员初始化列表中进行

```
int f;  
class CDemo {  
    private:  
        const int num; //常量型成员变量  
        int & ref;      //引用型成员变量  
        int value;  
    public:  
        CDemo(int n): num(n), ref(f), value(4) { }  
};  
int main(){  
    cout << sizeof(CDemo) << endl;  
    return 0;  
} //输出结果是: 12
```

主要内容

- 内联函数/内联成员函数
- 成员函数重载与参数缺省
- this 指针
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元



友元: friend

□ 友元分为友元函数和友元类两种

■ 友元函数: 一个类的友元函数可以访问该类的私有成员

class CCar ; //提前声明 CCar类, 以便后面的CDriver类使用

class CDriver{

public:

void ModifyCar(CCar * pCar) ; //改装汽车

};

class CCar{

private:

int price;

friend int MostExpensiveCar(CCar cars[], int total); //声明友元

friend void CDriver::ModifyCar(CCar * pCar); //声明友元

```
void CDriver::ModifyCar( CCar * pCar ) {  
    pCar->price += 1000; //汽车改装后价值增加  
}  
int MostExpensiveCar( CCar cars[], int total ) {  
    //求最贵汽车的价格  
    int tmpMax = -1;  
    for( int i = 0; i < total; ++i )  
        if( cars[i].price > tmpMax )  
            tmpMax = cars[i].price;  
    return tmpMax;  
}  
int main(){  
    return 0;  
}
```

□ 可以将一个类的成员函数 (包括构造/析构函数)
说明为另一个类的友元

```
class B {  
    public:  
        void function();  
};  
  
class A {  
    friend void B::function();  
};
```

2. 友元类: 如果A是B的友元类, 那么A的成员函数可以访问B的私有成员

```
class CCar{  
    private:  
        int price;  
        friend class CDriver; //声明CDriver为友元类  
};  
class CDriver{  
    public:  
        CCar myCar;  
        void ModifyCar(){ //改装汽车  
            myCar.price += 1000; //因CDriver是CCar的友元类,  
        } //故此处可以访问其私有成员  
};  
int main(){ return 0; }
```

- 友元类之间的关系不能传递, 不能继承

本章小结

- 内联函数/内联成员函数
- 成员函数重载与参数缺省
- **this** 指针
- 静态成员
- 常量成员函数
- 成员对象和封闭类
- 友元



Thanks!



有类A如下定义：

```
class A {  
    int v;  
    public:  
        A (int n) { v = n; }  
};
```

下面哪条语句是编译不会出错的？

- A) A a1(3);
- B) A a2;
- C) A * p = new A();

有类A如下定义：

```
class A {  
    int v;  
    public:  
        A (int n) { v = n; }  
};
```

下面哪条语句是编译不会出错的？

A) A a1(3);

B) A a2;

C) A * p = new A();

假设A 是一个类的名字, 下面哪段程序不会调用A的复制构造函数?

A) **A a1, a2; a1 = a2;**

B) **void func(A a) { cout << "good" << endl; }**

C) **A func() { A tmp; return tmp; }**

D) **A a1; A a2(a1);**

假设A 是一个类的名字, 下面哪段程序不会调用A的复制构造函数?

A) A a1, a2; a1 = a2;

B) void func(A a) { cout << "good" << endl; }

C) A func() { A tmp; return tmp; }

D) A a1; A a2(a1);

假设 A 是一个类的名字，下面的语句生成了几个类 A 的对象？

```
1 A * arr[4] = { new A(), NULL, new A() };
```

- ☐ A 1
- ☐ B 2
- ☐ C 3
- ☐ D 4

假设 A 是一个类的名字，下面的语句生成了几个类 A 的对象？

```
1 A * arr[4] = { new A(), NULL, new A() };
```

☐ A 1

☒ B 2

☐ C 3

☐ D 4

```
1  class A {  
2      int v;  
3  public:  
4      A(int i) {  
5          v = i;  
6      }  
7      A() { }  
8  };
```

下面哪段程序不会引发类型转换构造函数被调用？

- Ⓐ A a1(4);
- Ⓑ A a3; a3 = 9;
- Ⓒ A a2 = 4;
- Ⓓ A a1,a2; a1 = a2;

```
1  class A {  
2      int v;  
3  public:  
4      A(int i) {  
5          v = i;  
6      }  
7      A() { }  
8  };
```

下面哪段程序不会引发类型转换构造函数被调用？

- Ⓐ A a1(4);
- Ⓑ A a3; a3 = 9;
- Ⓒ A a2 = 4;
- Ⓓ A a1,a2; a1 = a2;

```
1  int main() {  
2      A * p = new A[2];  
3      A * p2 = new A;  
4      A a;  
5      delete [] p;  
6  }
```

假设 A 是一个类的名字，下面的程序片段会类 A 的调用析构函数几次？

- ☐ A 1
- ☐ B 2
- ☐ C 3
- ☐ D 4

```
1  int main() {  
2      A * p = new A[2];  
3      A * p2 = new A;  
4      A a;  
5      delete [] p;  
6  }
```

假设 A 是一个类的名字，下面的程序片段会类 A 的调用析构函数几次？

- ☐ A 1
- ☐ B 2
- ☒ C 3
- ☐ D 4